Stencil Computations on Tenstorrent Wormhole

Lorenzo Piarulli
piarulli@di.uniroma1.it
Sapienza University of Rome
Rome, Italy

Daniele De Sensi
desensi@di.uniroma1.it
Sapienza University of Rome
Rome, Italy

Abstract

As investment in AI-focused accelerators grows and their deployment in supercomputing facilities expands, understanding whether these architectures can efficiently support traditional scientific kernels is critical for the future of High-Performance Computing. We investigate the mapping of 2D 5-point stencil computations onto the Tenstorrent Wormhole, a RISC-V AI dataflow accelerator. We develop two heterogeneous implementations: *Axpy*, which decomposes the stencil into element-wise submatrix operations, and *MatMul*, which reformulates it as a matrix multiplication. While the CPU baseline remains 3× faster end-to-end, profiling reveals that the isolated Wormhole kernel is competitive with CPU execution, with the gap driven by PCIe transfers, device initialization, and host-side preprocessing. Despite slower runtime, *Axpy* achieves lower energy consumption than the CPU baseline for large inputs. Through detailed profiling and theoretical analysis, we identify key architectural and software limitations of the current platform and outline concrete hardware and software directions that could make AI accelerators competitive for HPC workloads.

1 Introduction

High-Performance Computing (HPC) methods are fundamental for scientific discovery, from weather forecasting and climate modeling to computational genomics and materials science [9, 17]. Many of these scientific challenges are modeled using Partial Differential Equations (PDEs), which require iterative approximation algorithms that are computationally intensive, frequently demanding large-scale computations running for days or weeks on supercomputers.

At the same time, the explosive growth of AI workloads has redirected supercomputing resources toward deep learning. This trend is driving substantial investment in domain-specific accelerators (DSAs) optimized for matrix-heavy AI tasks [11], and it is increasingly likely that future HPC systems will dedicate large partitions to these accelerators. This raises a critical question: can these AI-focused architectures also serve traditional HPC kernels such as stencil computations, or will the scientific computing community be left behind?

Tenstorrent Wormhole [14] exemplifies this new class of accelerators. It is a RISC-V-based design built around a grid of Tensix cores, each executing dataflow operations on fixed 32×32 tiles, primarily designed for GEMM-based AI workloads. Unlike GPUs with mature kernel-level ecosystems (CUDA, ROCm), Wormhole represents an evolving architecture with a kernel-level programming model (TT-Metalium) that gives developers direct control over hardware resources.

In this paper, building on Brown et al.'s work on Tenstorrent Grayskull [4], we develop two heterogeneous CPU–Wormhole stencil implementations and conduct an extensive analysis of their performance, energy efficiency, and architectural bottlenecks.

Our approach adopts a heterogeneous execution model that performs frequent data transfers and CPU-side format conversions at every iteration, diverging from conventional accelerator best practices. This choice stems from a practical constraint: Brown and Barton's work on Grayskull [4] demonstrated that performing scalar operations entirely on-device is prohibitively expensive due to the Tensix cores' lack of efficient scalar support, and we were unable to find an alternative on-device solution for Wormhole. We therefore opted for a CPU–accelerator work division, accepting the performance penalty of repeated transfers in exchange for a working implementation that could reveal architectural and API limitations and suggest concrete directions for future hardware and software improvements. In addition, this model also evaluates unified memory architectures, such as AMD MI300A [3] or NVIDIA GH200 [10], where transfer penalties disappear and hybrid execution scheme becomes more efficient.

Our contributions are as follows:

1. Two heterogeneous methodologies which we denote as *Axpy* and *MatMul*, for mapping 2D 5-point stencil computations onto Wormhole, with a clear analysis of each method's limitations.
2. A detailed characterization of Wormhole's limitations through profiling with Tracy, identifying bottlenecks on both the hardware and software side and proposing architectural and API improvements to enable efficient HPC computation on future Tenstorrent accelerators.
3. A theoretical analysis of Unified Virtual Memory (UVM) and Unified Physical Memory (UPM) scenarios showing that unified memory could make the heterogeneous approach competitive, suggesting this as a promising architectural direction. We further discuss how Tenstorrent's Blackhole architecture could address the identified limitations for future HPC workloads on Tenstorrent hardware.

2 Related Work

HPC on Tenstorrent accelerators. Brown and Barton [4] presented the first systematic evaluation of stencil computations on the Tenstorrent Grayskull architecture. Their *Axpy*-style approach demonstrated higher energy efficiency than conventional CPUs but was limited by Grayskull's LPDDR memory bandwidth and the inefficiency of scalar operations on the lightweight Tensix RISC-V cores. More recently, Brown et al. [5] ported the Cooley–Tukey FFT algorithm to Wormhole, demonstrating that while the accelerator is slower than a 24-core Xeon Platinum CPU, it draws approximately 8× less power and consumes 2.8× less energy. Amati et al. [2] accelerated gravitational N-body simulations on the Wormhole n300, achieving more than 2× speedup and approximately 2× energy savings compared to optimized CPU implementations. These works collectively indicate that Tenstorrent accelerators, while not yet performance-competitive for all HPC workloads, offer compelling energy efficiency advantages.

Stencils on tensor cores. Chen et al. [7] proposed ConvStencil, which transforms stencil computations into matrix multiplications to leverage GPU tensor cores. Their approach demonstrated significant speedups on NVIDIA GPUs by reformulating the stencil as an im2col-style transformation followed by GEMM. Our *MatMul* method draws inspiration from this work but adapts it to Wormhole's tiled execution model, which imposes stricter alignment constraints than GPU tensor cores.

Architectural characterization. Cai et al. [6] assessed Tenstorrent's matrix multiply acceleration capabilities, providing early insights into the architecture's strengths and limitations. A recent microbenchmarking study [1] dissected the Blackhole architecture empirically, confirming that the Tensix dataflow model achieves high utilization of internal bandwidth when computation and communication are properly overlapped.

3 Background

3.1 Stencil Computations

Stencil computations are foundational HPC kernels used to solve PDEs through finite-difference methods (FDM). In this work, we solve the 2D Laplace equation $\Delta u = 0$ using the Jacobi iterative method. The discretized update rule for the 5-point stencil is:

$$u_{i,j}^{(k+1)} = \frac{1}{4}\left(u_{i+1,j}^{(k)} + u_{i-1,j}^{(k)} + u_{i,j+1}^{(k)} + u_{i,j-1}^{(k)}\right) \tag{1}$$

where (i, j) denotes the grid position and k the iteration index. This formulation applies Dirichlet boundary conditions (zero-valued boundaries) and iterates for a fixed number of iterations rather than until convergence. Although conceptually simple, efficiently mapping this pattern onto tile-based architectures requires careful data layout management since the stencil accesses neighboring elements that may span different tiles.

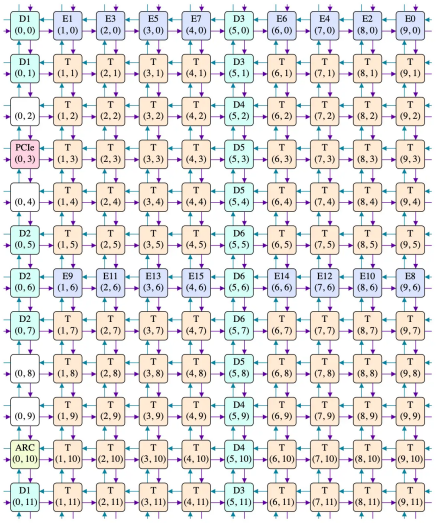


Figure 1. Tenstorrent Wormhole architecture. The 10×12 grid contains 64 Tensix cores (T), DRAM controllers (D), Ethernet interfaces (E), PCIe controller, and ARC management core (image from [8]).

3.2 Tenstorrent Wormhole Architecture

Tenstorrent Wormhole is a RISC-V AI accelerator available as a PCIe card (n150d/n300d variants). The architecture, shown in Figure 1, is built around a mesh of 64 Tensix cores interconnected through a high-bandwidth Network-on-Chip (NoC), complemented by DRAM banks and Ethernet interfaces for multi-accelerator scaling.

Each Tensix core (Figure 2) is a self-contained dataflow processor integrating five lightweight "baby" RISC-V CPUs, a matrix engine (FPU) optimized for 32×32 tile operations, a vector/SIMD unit (SFPU), pack/unpack units for data format conversion, two NoC interfaces, and approximately 1.5 MB of local SRAM. The five RISC-V cores serve distinct roles: two handle data movement (NoC reader/writer), while three manage computation through a pipelined Unpack→Math→Pack paradigm. The cores synchronize through a shared destination register (DST) using explicit lock/release primitives.

A critical constraint for HPC workloads is that Wormhole operates natively on 32×32 tiles. Although smaller tile sizes are theoretically supported, key API functions such as `tilize_nfaces()` and `untilize_nfaces()` currently operate only on 32×32 tiles. Furthermore, all data transferred to the accelerator must be converted from row-major layout to a tiled memory layout (`tilize`), and results must be converted back (`untilize`). The TT-Metalium programming model provides kernel-level access to hardware resources through C++ APIs compiled Just-In-Time at runtime. All

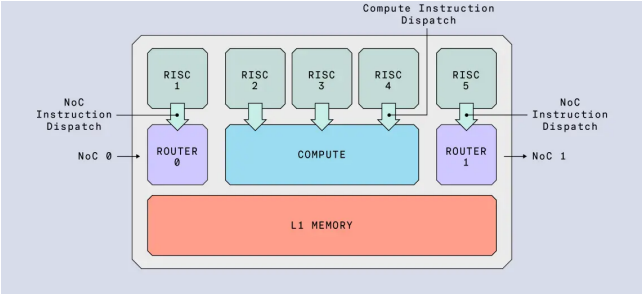


Figure 2. Tensix core architecture. Five baby RISC-V cores coordinate NoC data movement (Router 0/1), computation (Compute block with matrix and vector engines), and local L1 memory access (image from [15]).

Table 1. Tenstorrent Wormhole n150d specifications.

Specification	Value
Tensix Cores	72 (64 usable)
Clock	1 GHz
SRAM	108 MB
Memory	12 GB GDDR6
Memory Bandwidth	288 GB/s
TeraFLOPS (FP8)	262
TeraFLOPS (FP16)	74
Total Board Power	160 W
System Interface	PCIe Gen4 ×16

experiments in this work use TT-Metal version 0.62.2 with *bfloat16* precision.

Table 1 summarizes the key specifications of the Wormhole n150d used in our experiments.

4 Methodology

4.1 Heterogeneous Execution Model

Both of our methodologies follow a heterogeneous CPU–accelerator execution model that deliberately diverges from conventional accelerator usage. Typically, data is transferred to an accelerator once, all iterations execute on-device, and only the final result is transferred back. However, this model is not always feasible due to architectural or software limitations. Consequently, results must be transferred back to the CPU after each iteration for scalar preprocessing; this introduces significant data movement overhead before they are sent back for the subsequent iteration.

This design choice is motivated by one factor: **Wormhole's inability to efficiently handle scalar operations**. The architecture excels at tile-based pipelined MIMD operations but is highly inefficient for scalar and irregular computations. The baby RISC-V cores within each Tensix are lightweight processors not optimized for intensive scalar work [4]. Performing data extraction and boundary handling on the accelerator, as attempted by Brown et al. on

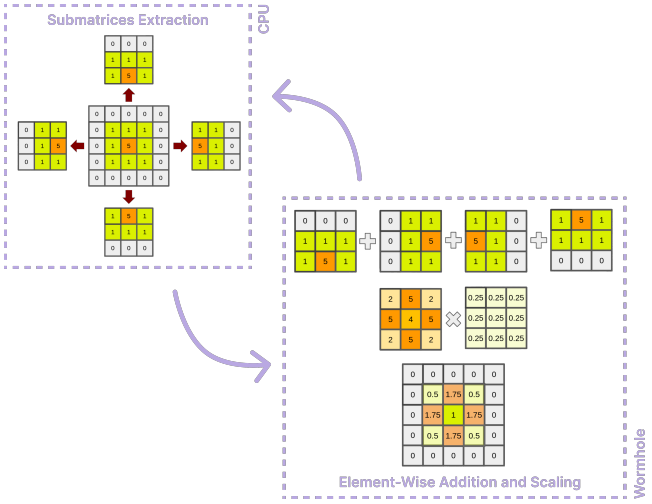


Figure 3. *Axpy* pipeline. From the padded input the submatrices are extracted, the submatrices data are sent to Wormhole for element-wise addition and scaling.

Grayskull [4], proved extremely expensive due to the architecture's poor scalar performance and strict memory alignment constraints. Conversely, CPUs are optimized for scalar manipulations, cache-friendly data movement, and control-heavy operations.

Both implementations are therefore structured in two phases per iteration: data *preprocessing* and *computation*. In the *Axpy* method (Sec. 4.2), the CPU extracts the shifted submatrices (*data preprocessing*), while the Tenstorrent accelerator computes the element-wise sum and scaling (*computation*). In the *MatMul* method (Sec. 4.3), the CPU converts the input to stencil-to-row format (*data preprocessing*), and the Tenstorrent accelerator performs the matrix multiplication (*computation*). At the end of each iteration, the result is transferred back to the CPU before the next iteration begins.

4.2 *Axpy* Method

Inspired by Brown's Grayskull work [4], we decompose the 5-point stencil into submatrix operations (Figure 3).

The method proceeds as follows for each iteration:

CPU phase. From the padded input matrix, extract four shifted submatrices corresponding to the stencil neighbors: up (shift one row upward), down (shift one row downward), left (shift one column left), and right (shift one column right). Each submatrix is stored in a contiguous buffer and padded so that its total element count is divisible by $32 \times 32 = 1024$, satisfying Wormhole's tile alignment. In the first iteration, the input is padded with Dirichlet boundary conditions (halo of zeros). In subsequent iterations, padding and extraction are fused into a single optimized routine.

Wormhole phase. The four submatrix buffers are transferred over PCIe to Wormhole. Tiles are distributed across all 64 Tensix cores: for an $N \times N$ matrix with tile size 32,

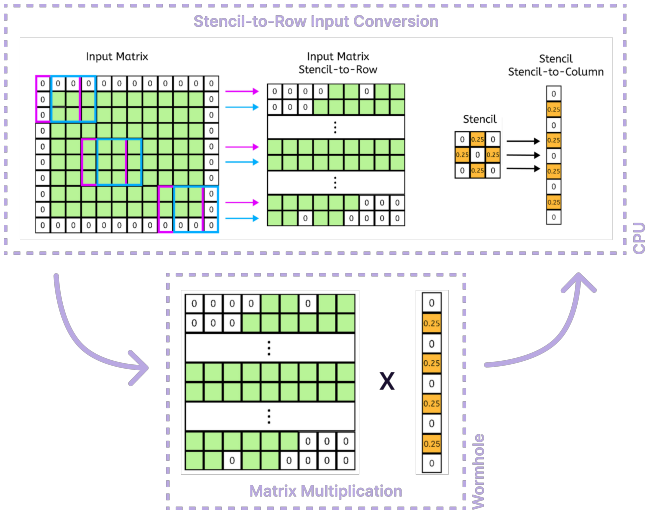


Figure 4. *MatMul* pipeline. The padded input is converted to stencil-to-row format, the stencil kernel is flattened to a column vector, both are aligned to 32×32 tiles via padding, and the tilized data is sent to Wormhole for matrix multiplication.

the total number of tiles is $T = (N/32)^2$, and each core receives $\lceil T/64 \rceil$ tiles. For each tile index k, the compute kernel executes:

$$\text{out}_k^{32\times32} = 0.25^{32\times32} \odot (\text{up}_k^{32\times32} + \text{down}_k^{32\times32} + \text{left}_k^{32\times32} + \text{right}_k^{32\times32}) \quad (2)$$

using the matrix engine for element-wise addition and element-wise product (denoted as $\odot$). The scalar 0.25 is represented as a constant 32×32 tile for compatibility with the matrix engine. Results are written back to device DRAM and then transferred to CPU DRAM over PCIe.

The key advantage of *Axpy* is that data remains in row-major layout throughout; no `tilize`/`untilize` conversion is needed because element-wise addition is layout-agnostic. The only requirement is that buffer sizes are multiples of the tile element count.

Limitations. The fixed 32×32 tile size mandates padding for arbitrary input dimensions, wasting memory and compute cycles on padding elements. Furthermore, the per-iteration CPU–device round-trip over PCIe Gen4 (≈31.5 GB/s per direction) introduces transfer overhead that dominates.

4.3 *MatMul* Method

In our second methodology, inspired by ConvStencil [7], we reformulate the stencil as a matrix multiplication (Figure 4). This leverages Wormhole's highly optimized matrix multiplication engine.

CPU phase. The padded input is converted to stencil-to-row format (denoted as *In*): for each grid point, the 3×3 neighborhood is unrolled into a row of 9 elements. An $N \times N$ grid produces an $(N^2) \times 9$ matrix. The stencil weights are flattened into a 9×1 column vector (denoted as *St*). Both

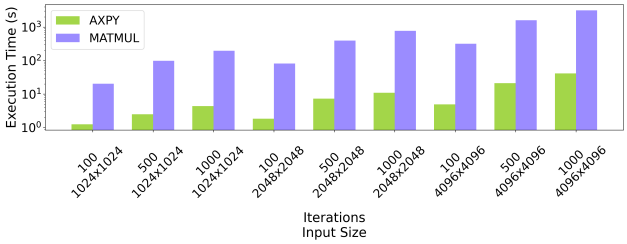


Figure 5. Execution time comparison between *Axpy* and *MatMul* heterogeneous implementations (log scale). *Axpy* is approximately 75× faster across all configurations.

matrices must then be aligned to 32×32 tiles by padding: the column vector is padded to 32×1 and replicated 32 times to form a 32×32 tile; the input matrix rows are padded from 9 to 32 columns. Finally, `tilize_nfaces()` converts the row-major data to Wormhole's tiled memory layout.

Wormhole phase. Tiles are distributed across Tensix cores and tiled matrix multiplication is executed, using the batched matrix multiplication paradigm. For each tile index k:

$$\text{out}_k^{32\times32} = \text{In}_k^{32\times32}\text{St}^{32\times32} \quad (3)$$

After completion, `untilize_nfaces()` converts results back to row-major layout, and the CPU extracts the final grid for the next iteration.

Limitations. The *MatMul* method incurs severe overhead from three sources. First, the stencil-to-row transformation expands an 8×8 matrix (128 B in FP16) to 4096 B after tiling, a 32× memory increase. Second, the `tilize`/`untilize` conversions are performed by CPU-side utility functions that account for approximately 90% of total CPU-side execution time. Third, the larger memory footprint limits the maximum feasible input size: while *Axpy* handles up to 30720×30720, *MatMul* saturates DRAM at 16384×16384.

5 Experimental Results

5.1 Experimental Setup

Experiments were conducted on a node of the Sapienza Computer Science Department cluster equipped with two AMD EPYC 7301 16-core processors (64 virtual cores total), 256 GB DRAM, and a Tenstorrent Wormhole n150d accelerator. All code was compiled with Clang 17 at `-O3` with `-march=native`. Problem sizes range from 1024^2 to 30720^2 elements in bfloat16 precision, with iteration counts of 100, 500, and 1000. The CPU baseline is an OpenMP-parallelized stencil implementation with SIMD optimizations enabled via compiler flags.

5.2 *Axpy* vs. *MatMul* Comparison

Figure 5 compares the two heterogeneous implementations. *Axpy* outperforms *MatMul* by approximately 75× across all tested configurations. The gap widens with input size,

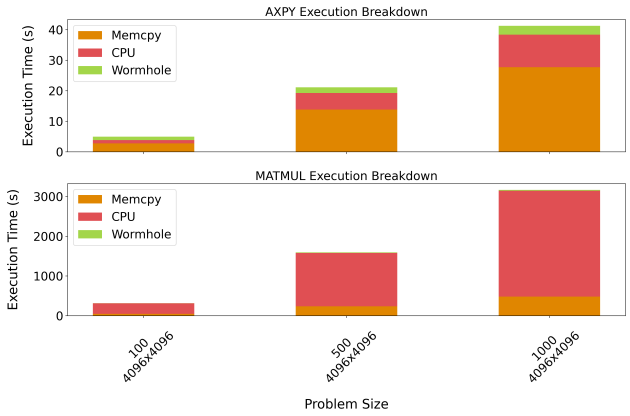


Figure 6. Execution time breakdown by phase. Top: *Axpy* shows balanced distribution across CPU, memcpy, and Wormhole. Bottom: *MatMul* is dominated by CPU-side tiling conversions (≈90%).

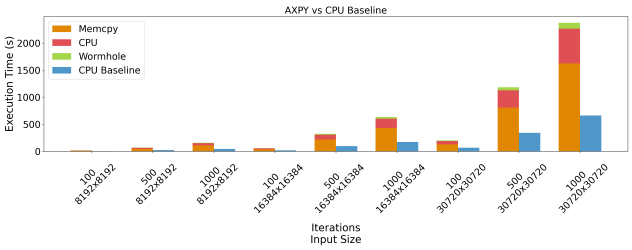


Figure 7. Execution time comparison between *Axpy* and CPU baseline implementations.

consistent with the quadratic growth of the stencil-to-row expansion in *MatMul*.

The phase breakdown in Figure 6 reveals the root cause. In *Axpy*, execution time is distributed across CPU tasks (15%), memory copies (25%), and Wormhole computation (60%), indicating a reasonable balance. In contrast, *MatMul* spends approximately 90% of its time on CPU-side work—dominated by the `tilize_nfaces()` and `untilize_nfaces()` functions—with only 5% each for memory copies and Wormhole computation. The Wormhole engine is thus largely idle in the *MatMul* variant, negating the advantage of using optimized matrix multiplication hardware.

5.3 *Axpy* vs. CPU Baseline

Given *Axpy*'s clear superiority, we evaluate it against the optimized CPU baseline. The CPU baseline remains approximately 3× faster than the heterogeneous *Axpy* implementation (Figure 7). This gap is primarily attributed to two factors: PCIe transfer overhead and CPU-side submatrix extraction repeated at every iteration (Figure 7).

However, profiling with the Tracy profiler [12] reveals a critical insight. When we isolate the actual kernel execution time on Wormhole (excluding device initialization, kernel launch, and transfer overhead), the Wormhole kernel is *competitive* with the CPU baseline for the *Axpy* variant. Table 2 quantifies this: the host-observed execution time includes a near-constant overhead of approximately 1s from device initialization that does not scale with input size. For large inputs (1024^2 at 1000 iterations), the pure kernel time is only 124 ms, while the host-observed time is 1376 ms—an overhead factor exceeding 10×.

Table 2. Kernel execution time: isolated (Tracy-profiled) vs. host-observed (including initialization and transfer overhead).

Config (iter-input)	Kernel Time	Total Time
100 – 128^2 (*Axpy*)	0.50 ms	1006 ms
1000 – 128^2 (*Axpy*)	4.96 ms	1140 ms
100 – 1024^2 (*Axpy*)	12.6 ms	981 ms
1000 – 1024^2 (*Axpy*)	124 ms	1376 ms
100 – 128^2 (*MatMul*)	2.58 ms	1013 ms
1000 – 1024^2 (*MatMul*)	1358 ms	2460 ms

The Tracy-profiled pipelining analysis further confirms that the dataflow paradigm within each Tensix core functions as designed: the Unpack thread prepares data, the Math thread executes arithmetic, and the Pack thread stores results, with effective overlap across tiles. The Reader and Writer threads stream data concurrently, and all cores exhibit similar timing patterns due to balanced workload distribution and a shared 1 GHz clock domain. The dominant bottleneck is not on-chip computation but rather the surrounding infrastructure: device initialization, PCIe transfers, and CPU-side preprocessing.

5.4 Energy Efficiency

Energy efficiency is a compelling advantage of the Wormhole architecture. Using TT-SMI for Wormhole power measurements and the AMD EPYC 7301's TDP (170 W) as a CPU estimate, we computed energy as $E = \text{Runtime} \times \text{Power}\ (P)$. Wormhole's idle power is approximately 11 W, rising to 20–24 W during computation. Despite being 3× slower, the heterogeneous *Axpy* implementation consumes *less total energy* than the CPU baseline for large inputs at high iteration counts, if we remove the data movement energy consumption. This advantage stems from Wormhole's architectural characteristics: high core count for balanced workload distribution and the modest 1 GHz operating frequency.

When isolating only the Wormhole kernel execution (excluding CPU preprocessing), the energy advantage is even more pronounced: the kernel operates at 20–24 W for milliseconds, whereas the CPU baseline sustains 170 W for comparable or longer durations. This finding suggests that with

architectural improvements to reduce transfer and conversion overhead, Wormhole-class accelerators could be highly energy-efficient HPC platforms.

6 Discussion

6.1 Identified Limitations

Our analysis reveals four critical architectural and software limitations of the current Wormhole platform for HPC workloads:

Fixed 32×32 tiling constraint. All buffers must be aligned to 32×32 tiles, precluding fine-grained computation. PDE solvers frequently require small vector operations for boundary handling and neighbor access. Although the hardware is capable of using smaller tiles, the API (`tilize_nfaces()`, `untilize_nfaces()`) does not support them. Enabling flexible tile sizes would significantly reduce padding overhead and memory waste.

Inefficient scalar and small-vector support. Wormhole's baby RISC-V cores are not designed for intensive scalar workloads. This forces format conversions, boundary handling, and data extraction to be offloaded to the host CPU, creating an inherent bottleneck. The *MatMul* variant's 90% CPU time is a direct consequence: the stencil-to-row transformation and tiling conversion are scalar-heavy operations that Wormhole cannot execute efficiently.

Tilize/untilize overhead. The CPU-side `tilize_nfaces()` and `untilize_nfaces()` functions introduce prohibitive latency for any methodology requiring tiled data layout. The *Axpy* method avoids this entirely because element-wise operations are layout-agnostic, but any approach leveraging Wormhole's matrix multiplication engine must pay this cost. Hardware support for flexible memory layouts, or on-chip tiling engines, would be transformative.

PCIe Gen4 bottleneck. The discrete CPU–accelerator design imposes ≈31.5 GB/s per-direction bandwidth through PCIe Gen4 ×16. For our iterative methodology, this cost is paid every iteration, dominating total execution time for moderate input sizes. Tighter CPU–accelerator integration through high-bandwidth interconnects or unified memory would dramatically reduce this overhead.

6.2 Unified Memory Analysis

To quantify the potential of future architectures, we model two unified memory scenarios (Figure 8).

UVM model. We replace PCIe Gen4 bandwidth (≈31.5 GB/s) with NVLink-C2C-class bandwidth (≈450 GB/s), as found in the NVIDIA GH200 Grace Hopper Superchip [10]. This reduces transfer overhead by approximately 15×. Under this model, the *Axpy* method approaches CPU baseline performance while maintaining significantly lower energy consumption.

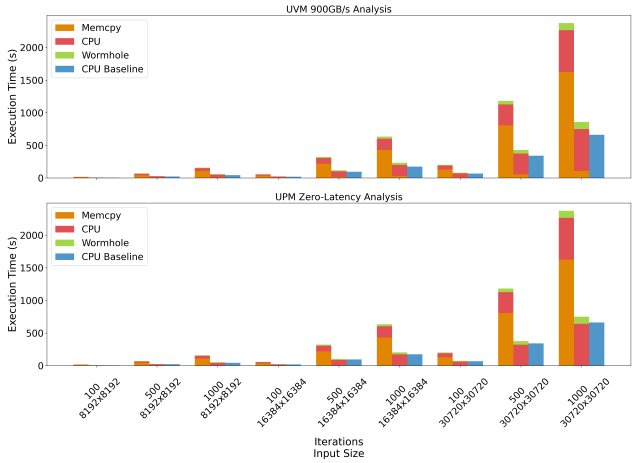


Figure 8. Theoretical performance under UVM (top, 900 GB/s NVLink-C2C bandwidth with 450 GB/s per-direction) and UPM (bottom, zero transfer overhead) scenarios compared to the CPU baseline.

UPM model. We model complete memory unification, as in AMD MI300A [3], where CPU and accelerator share coherent physical memory. This eliminates transfers entirely and also removes the need for tiling functions (`tilize`/`untilize`), which account for ≈90% of *MatMul*'s CPU time. Under UPM, the *Axpy* method could match or exceed CPU baseline performance at a fraction of the energy consumption. Even the *MatMul* method becomes viable once the dominant conversion overhead is eliminated.

An additional benefit of APU-based architectures, not captured in these projections, is the reduction of accelerator initialization overhead. Integrating CPU and accelerator within the same package minimizes the near-constant ≈1 s startup cost observed in our experiments.

6.3 Future Directions: Tenstorrent Blackhole

Tenstorrent's next-generation Blackhole architecture [13] addresses several of the limitations identified in this work and represents a significant step toward making RISC-V AI accelerators viable for HPC workloads.

The most transformative change is the inclusion of 16 SiFive x280 64-bit, dual-issue, in-order CPU cores arranged in four clusters [1, 16]. These cores are sufficiently powerful to boot Linux and serve as an on-device host, fundamentally transforming Blackhole from a traditional PCIe accelerator requiring host supervision into a *standalone AI computer*. For our stencil workloads, this means that the scalar-heavy preprocessing (submatrix extraction, boundary handling, and format conversions) could be executed on the Big RISC-V cores directly on-chip, without PCIe round-trips. The Big cores share 64 MB of total L2 cache and have coherent access

to 32 GB of GDDR6 memory through a dedicated interconnect separate from the Tensix NoC [1].

In addition, Blackhole scales to 120 Tensix cores (up from 64 on Wormhole), with 1.5 MB SRAM per core totaling 180 MB of distributed on-chip memory [13]. This increased parallelism and local storage could better hide transfer latencies and reduce the need for off-chip memory access during stencil iterations. It moves to PCIe Gen5 ×16, doubling available bandwidth compared to Wormhole's Gen4 interface. In addition, the transition from 12 nm (Wormhole) to 6 nm enables higher transistor density and improved power efficiency, supporting the increased core count within a 300 W power envelope. With Big RISC-V cores handling scalar preprocessing on-chip and 120 Tensix cores executing tiled computation, Blackhole approximates the UPM scenario analyzed in Section 6: data movement between the "CPU" and "accelerator" portions occurs over high-bandwidth on-chip interconnects rather than PCIe. While Blackhole does not implement true UPM (the Big cores and Tensix cores access separate memory hierarchies), the tight integration within a single chip eliminates the dominant bottlenecks we identified. Reassessing our methodologies on Blackhole is a clear and immediate direction for future work.

7 Conclusions and Future Work

We have presented a comprehensive analysis of mapping stencil computations onto the Tenstorrent Wormhole RISC-V AI accelerator through two heterogeneous CPU–accelerator methodologies. The *Axpy* method, which avoids tiled format conversions entirely, outperforms the *MatMul* approach by 75× and demonstrates lower energy consumption than a CPU baseline despite being 3× slower. Profiling reveals that the Wormhole kernel itself is competitive with CPU execution, since the performance gap is dominated by transfer overhead, device initialization, and CPU-side preprocessing.

Our theoretical analysis shows that unified memory architectures could close this gap: UVM reduces transfer overhead by 15×, while UPM eliminates it entirely. Tenstorrent's Blackhole architecture, with its 16 on-chip Big RISC-V cores capable of scalar computation and a standalone operating model, represents a concrete path toward realizing these benefits.

The most immediate direction for future work is evaluating our methodologies on Blackhole hardware, where the on-chip Big RISC-V cores could handle preprocessing without PCIe transfers. Beyond that, we plan to explore new algorithmic formulations to efficiently map stencils onto dataflow architectures, extend to multi-chip configurations leveraging Blackhole's Ethernet-based interconnect for distributed stencil computation, and investigate alternative PDE solvers to enable spectral-domain methods on Tenstorrent hardware.

References

[1] 2025. Dissecting the Tenstorrent Blackhole Architecture via Microbenchmarking. https://asplos.dev/wordpress/wp-content/uploads/2025/09/TT_bench-1.pdf. Accessed: 2025-11-01.

[2] Giorgio Amati, Matteo Turisini, Andrea Monterubbiano, Mattia Paladino, Elisabetta Boella, Daniele Gregori, and Danilo Croce. 2025. Accelerating Gravitational N-Body Simulations Using the RISC-V-Based Tenstorrent Wormhole. In *Proceedings of the SC'25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM. doi:10.1145/3731599.3767528

[3] AMD Corporation. 2023. AMD Instinct MI300A APU Product Overview. https://www.amd.com/en/products/accelerators/instinct/mi300a.html. Accessed: 2025-10-05.

[4] Nick Brown and Ryan Barton. 2024. Accelerating Stencils on the Tenstorrent Grayskull RISC-V Accelerator. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1690–1700. doi:10.1109/SCW63240.2024.00211

[5] Nick Brown, Jake Davies, and Felix Le Clair. 2025. Exploring Fast Fourier Transforms on the Tenstorrent Wormhole. In *ISC 2025 Workshops (Lecture Notes in Computer Science)*. Springer. arXiv:2506.15437.

[6] Z. Cai, R. Giordano, et al. 2023. Assessing Tenstorrent's RISC-V Matrix Multiply Acceleration. *arXiv preprint arXiv:2305.10314* (2023).

[7] Yuetao Chen, Kun Li, Yuhao Wang, Donglin Bai, Lei Wang, Lingxiao Ma, Liang Yuan, Yunquan Zhang, Ting Cao, and Mao Yang. 2024. ConvStencil: Transform Stencil Computation to Matrix Multiplication on Tensor Cores. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '24)*. ACM, 333–347.

[8] corsix. 2024. Community Highlight: Tenstorrent Wormhole Series Part 2: Which disabled rows? Tenstorrent Newsroom. https://tenstorrent.com/newsroom/community-highlight-tenstorrent-wormhole-series-part-2-which-disabled-rows Accessed: 2024-11-18.

[9] P. Giannozzi, O. Baseggio, P. Bonfà, D. Brunato, R. Car, I. Carnimeo, C. Cavazzoni, S. De Gironcoli, P. Delugas, F. Ferrari Ruffino, et al. 2020. Quantum ESPRESSO Toward the Exascale. *The Journal of Chemical Physics* 152, 15 (2020).

[10] NVIDIA Corporation. 2023. NVIDIA GH200 Grace Hopper Superchip. https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/. Accessed: 2025-10-05.

[11] J. Schmidhuber. 2015. Deep Learning in Neural Networks: An Overview. *Neural Networks* 61 (2015), 85–117.

[12] Bartosz Taudul. 2024. Tracy Profiler. https://github.com/wolfpld/tracy.

[13] Tenstorrent. 2025. Blackhole Accelerator Specifications. https://docs.tenstorrent.com/aibs/blackhole/specifications.html. Accessed: 2025-11-01.

[14] Tenstorrent. 2025. Wormhole Accelerator Specifications. https://docs.tenstorrent.com/aibs/wormhole/specifications.html. Accessed: 2025-09-11.

[15] Tenstorrent Inc. 2024. *Tensix Neo: RISC-V-based AI IP For Extraordinary AI Performance*. Tenstorrent. https://tenstorrent.com/ip/tensix-neo Accessed: 2024-11-18.

[16] Jasmina Vasiljevic and Davor Capalija. 2024. Blackhole & TT-Metalium: The Standalone AI Computer and Its Programming Model. In *Hot Chips 36*.

[17] A. Zeni, G. Guidi, M. Ellis, N. Ding, M. D. Santambrogio, S. Hofmeyr, A. Buluç, L. Oliker, and K. Yelick. 2020. Logan: High-Performance GPU-Based X-Drop Long-Read Alignment. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 462–471.